

Let's revisit the FilterLock algorithm:

```
class Filter implements Lock {
    int[] level;
    int[] victim;

    public Filter(int n) {
        level = new int[n];
        victim = new int[n]; // use 1..n-1
        for (int i = 0; i < n; i++) {
            level[i] = 0;
        }
    }

    public void lock() {
        int me = ThreadID.get();
        for (int i = 1; i < n; i++) { // attempt level i
            level[me] = i;
            victim[i] = me;
            // spin while conflicts exist
            while ((∃k != me) (level[k] >= i && victim[i] == me)) {}
        }
    }

    public void unlock() {
        int me = ThreadID.get();
        level[me] = 0;
    }
}
```

Lemma: For any $0 \leq k \leq n - 1$ there are at most $n - k$ threads at level k .

Proof: Induction on k .

- Base: $k = 0$.

All n threads in our system start at this level and no more threads are added.

- Step: there are at most $n - k + 1$ threads at level $k - 1$.

Assume that there are $n - k + 1$ threads at level k at some moment. That means that all threads from level $k - 1$ we able to proceed to level k .

Let A be the last thread that wrote to `victim[k]`. That implies that for any other thread B at level k holds

$$\text{write}_B(\text{victim}[k]) \rightarrow \text{write}_A(\text{victim}[k])$$

Because B writes to `level[B]` before writing to `victim[k]`, we get

$$\text{write}_B(\text{level}[B] = k) \rightarrow \text{write}_B(\text{victim}[k]) \rightarrow \text{write}_A(\text{victim}[k])$$

Because A reads `level[B]` after writing to `victim[k]`, we get

$$\text{write}_B(\text{level}[B] = k) \rightarrow \text{write}_B(\text{victim}[k]) \rightarrow \text{write}_A(\text{victim}[k]) \rightarrow \text{read}_A(\text{level}[B])$$

And finally, because of that relation, `read(level[B])` in thread A always returned $k - 1$ – a value greater than or equal to $k - 1$ – so A should never be able to pass through waiting loop. Contradiction.

□

Theorem: Filter lock satisfies mutual exclusion.

Proof: Critical section is, by definition, a section of code that can be executed by one or zero threads at a time.

From the lemma above we conclude that there is at most one thread at level $n - 1$. Because `lock` returns only when a thread has passed all levels, this guarantees that there is at most one thread in the critical section at any particular moment. □